Student No._____ **Name:**_____

특별한 언급이 없는 한, 32bit x86 프로세서의 protected mode 에서 MASM 을 사용함을 가정함

1. [5pt] 다음은 32bit x86 프로세서를 위한 Assembly Language 에 관련된 설명이다. **옳지 <u>않은</u>** 설명을 **<u>모두</u>** 고르시오. **답:**_____

① Assembly Language 는 기계(CPU)에 의해 직접 실행될 수 있는 언어이다.

② Assembly Language 는 Machine Language 와 1 대 1 대응관계에 있으므로 Assembly Language 로 작성된 코드의 각 LINE 은 그에 대응하는 하나의 Machine Language 로 번역된다.

③ Assembly Language 로 작성된 코드는 Assembler 에 의해 Assemble 되면 바로 실행 가능하다.

④ Assembly Language 와 비교했을 때, C, C++와 같은 High Level Language 들은 이식성은 떨어지지만 더 나은 실행 성능을 제공한다는 이점을 갖는다.

⑤ Assembly Language 를 공부하는 것은 컴퓨터의 동작 원리를 이해하는 것에 효과적이다.

2.1 [5pt] x86 시스템에서 부호있는 32 비트 정수의 범위를 쓰시오. (계산식으로 표현해도 됨)

최소		최대
(a)	~	(b)

2.2 [5pt] 2.1)의 (b)값이 EAX 레지스터에 저장되어 있다고 가정하자. ADD EAX, 1 을 수행한 다음 EAX 레지스터에 저장될 값을 Hexadecimal 로 쓰시오. (정확한 값을 쓰시오)

3. [10pt] CPU의 명령어 실행 사이클 5단계를 순서대로 설명하고, 필수적인 단계에 체크하시오.

단계	**필수 여부**	**간략한 설명** (누가 어디에서 무엇을 어떻게 하는지 등)
1		
2		
3		
4		
5		

4.1 [5pt] 폰 노이만 컴퓨터 구조에서의 주된 병목지점(Bottleneck)을 설명하시오

4.2 [5pt] 4.1)을 해결(완화)하기 위한 전략으로 널리 쓰이는 방법을 설명하시오.

5. [5pt] 다음은 x86 프로세서에 대한 설명이다. **옳지 <u>않은</u>** 설명을 **<u>모두</u>** 고르시오.

답:_____

① 프로세서의 진화 과정에서 이전 프로세서들과의 호환성 유지를 위해 다양한 operation 모드를 제공한다.

② Real-address 모드는 초창기 16 비트 x86 프로세서들에서 사용되었던 모드로써, 메모리를 비롯한 하드웨어로의 직접 접근을 허용한다.

③ Real-address 모드에서는 16 비트 세그먼트 값과 16 비트 오프셋 값을 결합하여 32 비트 주소를 구성하므로 4GB 의 주소공간을 형성한다.

④ Protected 모드는 메모리를 세그먼트라는 영역으로 구분짓고, 프로그램에게 주어진 세그먼트 내에서만 접근을 허용한다.

⑤ EIP 레지스터는 다음 실행할 명령어를 저장하고 있으며, 명령어의 종류에 따라 저장되는 명령어의 size 가 달라진다.

Student No._____ **Name:**_____

6. [5pt] 최근 출시한 일부 제조사들의 Android 기반 스마트폰들이 게임에서의 특수 효과를 위해 하드웨어에서 구현된 최신 기능을 제공한다고 가정하자. 하지만, Android 운영체제에서 이러한 기능들에 대해 아직 지원하지 않는다고 할 때, 해당 기능 제공을 위해 게임 개발자로써 어떠한 I/O Access Level을 선택해야 하는지, 그에 따른 장단점은 무엇이 있는지 설명하시오.

7.1 [7pt] 다음 어셈블리어 코드의 마지막 구문이 실행된 후, Carry Flag (CF), Overflow Flag (OF), Sign Flag (SF) 세팅 여부를 답하시오.

```
MOV AL, 180
MOV BL, -90
ADD AL, BL
```

CF	OF	SF

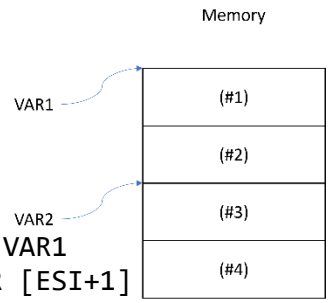
7.2 [8pt] 7.1 에서 CPU 가 그것을 어떻게 탐지하는지 그 계산 과정을 쓰고 설명하시오.

8. 다음 주어진 어셈블리어 프로그램들에 대한 물음에 답하시오.

(1) [5pt] 다음 MASM 프로그램이 있다. DATA1 과 DATA2 가 메모리에 그림과 같이 위치한다고 가정할 때, AX 레지스터에 저장될 값을 Hexadecimal 로 쓰시오.
(만약, 어셈블 과정에서 에러가 발생한다면, 어느 구문에서 왜 에러가 발생하는지 설명하시오.)

```
.data
VAR1 WORD -1
VAR2 SWORD 257

.code
MOV ESI, OFFSET VAR1
MOV AX, WORD PTR [ESI+1]
```



(2) [5pt] 다음 프로그램 실행 후, AL 과 AH 레지스터에 저장될 값을 Hexadecimal 로 쓰시오.
(만약, 어셈블 과정에서 에러가 발생한다면, 어느 구문에서 왜 에러가 발생하는지 설명하시오.)

```
.data
ARRAY WORD 123h,234h,345h,456h,789h

.code
MOV AL, BYTE PTR [ARRAY+2]
MOV AH, BYTE PTR [ARRAY+3]
```

(3) [10pt] 다음의 C/C++ 프로그램을 어셈블리어 프로그램으로 변환하도록 빈칸을 채우시오. (2 바이트의 int 를 쓰는 시스템 가정)

```
int arr[10] = { 20, 20, 20, 20, 20, 20, 20, 20, 20, 20 };
arr[6] = arr[5] + 15;
```

```
.data
arr WORD    (#1)

.code
MOV         (#2)
ADD         (#3)
MOV         (#4)
```

위치	명령어
(#1)	
(#2)	
(#3)	
(#4)	

(4) [10pt] 다음의 C/C++ 프로그램을 어셈블리어 프로그램으로 변환하도록 빈칸을 채우시오. (2 바이트의 int, 4 바이트의 long 을 쓰는 시스템 가정)

```
int var1;
long var2;

/*   */

var2 = (long)var1;
```

```
.data
VAR1    (#1)
VAR2    (#2)

.code
MOV     AX, VAR1
(#3)    VAR2, AX
```

위치	명령어
(#1)	
(#2)	
(#3)	

(5) [5pt] 다음 프로그램의 실행 결과 SUM 에 저장될 값을 예상하시오. (만약, 어셈블 과정에서 에러가 발생한다면, 어느 구문에서 왜 에러가 발생하는지 설명하시오.)

```
.data
ARRAY BYTE 10, 20, 30, 40, 50
SUM   BYTE 0

.code
   MOV ECX, LENGTHOF ARRAY
   MOV ESI, 0
L1:
   ADD SUM, ARRAY[ESI]
   INC ESI
   LOOP L1
```

(6) [5pt] 다음 프로그램에서 MySub 프로시저에 대한 call 명령이 실행된 직후 ESP 와 EIP 에 저장될 값을 예상하시오. (만약, 어셈블 과정에서 에러가 발생한다면, 어느 구문에서 왜 에러가 발생하는지 설명하시오.)

```
                main PROC
00000020          call MySub
00000025          mov  eax,ebx

                MySub PROC
00000040          mov eax,edx
                  .
                  .
                  ret
                MySub ENDP
```